

Ranking Structured Objects

Composable bijections, constrained rhythms and meaningful melody orders

Hob Nilre & Bo C. Herlin

2026-09-28

Abstract

A rank assigns an integer coordinate to an object; an unrank reconstructs that object from its coordinate. This article develops a compositional construction in which a space is divided into finite, ordered groups, each group is counted, and its component coordinates are combined. The same construction handles finite products, constrained lists, permutations and recursive tree shapes. Explicit pair inversion, reduced-ratio coordinates and labeled progression trees expose the role of counting and equality. For Fibonacci rhythms we prove an exact admission criterion and count and rank extensions that retain a chosen motif. A five-part limerick contour has four motif-preserving eight-part extensions among sixteen possible patterns. For melodies, grouping by total pitch travel gives a reversible coordinate whose increase never decreases that musical quantity. Exact counts, both inverse maps and a finite-instrument variant make this ordering reproducible. Worked calculations recover a list from rank 45 and a three-note melody from rank 36. We give the conditions that make both directions inverse, including object equality, disjoint decomposition, finite layers, compatible component domains and terminating recursion. Comparisons across implementations expose why plausible counts and successful round trips can still conceal an incomplete object domain. The result is a mathematical specification for reproducible enumeration and constrained musical exploration. The ordering guarantee concerns a declared musical measurement; its perceptual usefulness is a separate, testable question.

PDF created: 2026-09-30 18:55:01 UTC

Latest on GitHub: <https://github.com/hobnilre/code-rank>

1 An integer coordinate for a structured object

Suppose a search space consists of lists, arrangements of symbols or tree shapes. It is useful to ask for object number 45, return to exactly that object later, or assign disjoint intervals of objects to different workers. A sequence that produces the next object is one solution, but reaching a distant position may require constructing every predecessor. A reversible integer coordinate makes a different operation possible: count the alternatives that precede an object, and use those counts to locate or reconstruct it directly.

The central idea is simple. Divide the space into ordered, finite groups. Count each group without materializing its objects. Within a group, split an object into smaller parts whose ranks are already available. Addition accounts for skipped groups; mixed-radix arithmetic accounts for combinations of parts. Recursive application gives a coordinate for the whole object.

This article reconstructs that framework across the authors' Mathematica, Haskell, Java and Kotlin implementations and a current Rust implementation. The languages and interfaces change; the useful mathematical question persists: **under which conditions do counting, ranking and unranking describe exactly the same space?** The treatment gives explicit domains, both inverse laws, complete worked calculations and counterexamples to insufficient conditions.

A second question gives the musical applications their direction: **what should an increasing coordinate mean?** A reversible numbering can be arbitrary. A useful exploration order can instead begin with a declared measurement, such as total pitch movement, and count the alternatives at each value. We develop two concrete applications: exact access to Fibonacci rhythms that retain a specified motif, and melody coordinates ordered by a musically interpretable quantity.

Combinatorial ranking is an established subject; Wilf’s unified treatment already connects sequencing, ranking and selection [Wilf, 1977](#). A particularly close precedent is Feat, which constructs functional enumerations from finite parts using sums, products, guarded recursion and bijections, with applications to test generation [Duregård, Jansson and Wang, 2012](#). The contribution here is a self-contained synthesis, a contract analysis across the implementation family, and explicit constructions for constrained rhythm exploration and melodic motion. The admission theorem, motif counts and melody inverse below state what these applications establish. Historical priority for the underlying combinatorial operations is not a premise of those results.

2 What must be inverse?

Let X be a specified set of objects, with a specified meaning of equality. Write $\mathbb{N}_0 = \{0, 1, 2, \dots\}$ and $I_C = \{0, \dots, C - 1\}$ for a finite cardinality C , including $I_0 = \emptyset$. A dense rank and its unrank are maps

$$R : X \rightarrow I, \quad U : I \rightarrow X, \quad U(R(x)) = x, \quad R(U(r)) = r, \quad (1)$$

where $I = I_C$ for a finite space or $I = \mathbb{N}_0$ for a countably infinite one. The first identity says that every admissible object is recovered; the second says that every admissible coordinate is used exactly once. All ranks in this article are zero-based.

Equality is part of the specification. An ordered sequence of distinct symbols is different from an unordered set of those symbols. A tree shape is different from a tree carrying arbitrary labels. If labels are discarded, the inverse law can hold for shapes, but cannot recover the discarded labels. Likewise, the fractions $1/2$ and $2/4$ are distinct pairs and the same rational number. Choosing the object space comes before choosing its coordinate.

A finite count, a proof of countable infinitude, and an unknown total are three different kinds of information. An absent count in a software interface does not establish infinitude. Nor does an arbitrary-precision rank imply unbounded execution: grade limits, recursion limits and time budgets can still stop a mathematically valid request. Invalid coordinates and unfinished computation have different meanings.

3 The finite-group construction

3.1 Ordered groups and finite products

Assume an ordered decomposition

$$X = \bigsqcup_{g \geq 0} X_g, \quad X_g \cong \prod_{j=0}^{m_g-1} X_{g,j}, \quad C_g = \prod_{j=0}^{m_g-1} c_{g,j}, \quad S_g = \sum_{h < g} C_h, \quad (2)$$

where every $c_{g,j} = |X_{g,j}|$ is finite. The disjoint union means that every object has exactly one group. The displayed isomorphism requires mutually inverse splitting and assembly, not merely equal counts.

Fix ranks for the component spaces. In the **first component least significant** convention, an object $x \in X_g$, split into $(x_0, \dots, x_{m_g-1})$, receives

$$R(x) = S_g + \sum_{j=0}^{m_g-1} R_{g,j}(x_j) \prod_{i < j} c_{g,i}. \quad (3)$$

Thus the first component changes fastest when the local coordinate increases. A product with no factors has one object, the empty tuple, and local rank zero. A product with a zero-sized factor has no objects. Such a group is skipped; there is no division by its zero radix.

For a coordinate r , find the nonempty group with $S_g \leq r < S_g + C_g$. The residual $r - S_g$ has a unique mixed-radix expansion with digits $0 \leq d_j < c_{g,j}$. Successive remainders and quotients determine these digits. Applying each component unrank and then assembly reconstructs the object.

Theorem 1 (Compositional inverse). *Suppose the decomposition in (2) is disjoint and exhaustive, splitting and assembly are inverse, and every component rank satisfies (1). Then (3) and the reconstruction above are inverse on their stated domains. If the counts, group membership, component maps and assembly are computable, group lookup terminates for every valid coordinate. Recursive component constructions additionally require a well-founded descent.*

Proof. The intervals $[S_g, S_g + C_g)$ for nonempty groups are disjoint consecutive integer intervals. Within one interval, uniqueness of mixed-radix digits gives both local inverse identities. The component inverse laws and inverse assembly then give both identities for the whole object. Exhaustiveness supplies a finite-index group for every object. Every coordinate below the total count belongs to some such interval, so a sequential search reaches it after finitely many groups. If the total is infinite, the nondecreasing integer prefix sums are unbounded and the same argument applies to every $r \in \mathbb{N}_0$. Recursive use follows by induction on the stated descending measure. $\square$

This termination statement concerns valid coordinates. If a space is finite but its end is unknown, scanning infinitely many subsequent empty groups does not detect an out-of-range coordinate. A total count or an effective stopping certificate is needed for that rejection.

The opposite digit convention is equally valid. For two factors of sizes a and b , the first-component-most-significant formula is $br_0 + r_1$, whereas (3) gives $r_0 + ar_1$. In a 2×3 product, $(1, 0)$ has rank 3 under the former and rank 1 under the latter. A rank is meaningful only together with its ordering convention.

3.2 Infinite factors need finite layers

A product of infinite spaces cannot be inserted as a single finite group in (2). For $(a, b) \in \mathbb{N}_0^2$, group by $g = a + b$ and order each group by ascending a . Its count is $g + 1$, giving

$$R(a, b) = \frac{g(g+1)}{2} + a, \quad g = a + b. \quad (4)$$

The inverse can be written explicitly. Put $T_g = g(g+1)/2$. The rank r lies in the unique group with $T_g \leq r < T_{g+1}$. Solving $x(x+1)/2 = r$ for its nonnegative root, then rounding down, gives

$$g(r) = \left\lfloor \frac{\sqrt{8r+1} - 1}{2} \right\rfloor, \quad (5)$$

$$a(r) = r - T_{g(r)}, \quad b(r) = g(r) - a(r), \quad U(r) = (a(r), b(r)). \quad (6)$$

Indeed, $x(x+1)/2$ is strictly increasing for $x \geq 0$, so the group bounds are equivalent to $g \leq (\sqrt{8r+1} - 1)/2 < g+1$. They also give $0 \leq a(r) \leq g(r)$ and hence $b(r) \geq 0$. The unique group and residual prove both inverse identities. In particular $U(0) = (0, 0)$.

The author's 2007 mathematical notes contain this inverse and the example $r = 30000$: $g = 244$, $T_g = 29890$, and $U(30000) = (110, 134)$. Substitution into (4) returns 30000. This ranks ordered pairs: the equal fractions $110/134 = 55/67$ have different pair coordinates, 30000 and 7558. Distinct rational values require their own canonical domain below. For exact evaluation, set $q = \text{isqrt}(8r+1)$, where $\text{isqrt}(n) = \lfloor \sqrt{n} \rfloor$, and use $g = \lfloor (q-1)/2 \rfloor$. An integer square root avoids floating-point rounding at large triangular boundaries; arbitrary-precision arithmetic still has a cost.

Every pair is reached. By comparison, an order that exhausts all pairs with first coordinate zero before advancing to one gives later rows infinitely many predecessors; it cannot supply them finite dense ranks in that order.

More generally, a *grade* is a nonnegative integer attached to an object so that every fixed-grade class is finite. It need not measure physical size or quality. Its purpose here is to provide finite blocks with computable counts.

3.3 Choosing what an increasing rank means

Suppose a declared measurement $g : X \rightarrow \mathbb{N}_0$ has finite level sets. Arrange these sets by increasing g , and choose a fixed dense order within each nonempty set. The finite-group construction immediately gives

$$R(x) < R(y) \implies g(x) \leq g(y), \quad g(x) < g(y) \implies R(x) < R(y). \quad (7)$$

Indeed, each level occupies one consecutive interval, and all lower levels precede it. Ties in the measurement still need distinct coordinates. Moving within a level changes the object without increasing the measured quantity; displaying the measurement alongside the coordinate makes that distinction visible. Finiteness is substantive. If infinitely many stationary melodies differ only in an unrestricted starting pitch or duration, putting all of them before any moving melody prevents the latter from receiving a finite coordinate. Fixing, normalizing or finitely bounding those other dimensions makes the intended order possible. An alternative joint grade changes what “larger” means and must be declared accordingly. Equation (7) proves an ordering property; whether the chosen measurement matches a listener's impression requires its own evidence.

4 Counting compound spaces by grade

For a graded space A , write $c_A(g)$ for its count at grade g , with $c_A(g) = 0$ for negative arguments. Tagged alternatives are disjoint even when their payloads coincide. For finitely many alternatives with fixed nonnegative costs δ_i , and a two-factor product with cost δ , the counts are

$$c_{\text{sum}}(g) = \sum_i c_{A_i}(g - \delta_i), \quad (8)$$

$$c_{\text{product}}(g) = \sum_{h=0}^{g-\delta} c_A(h) c_B(g - \delta - h). \quad (9)$$

A sum with a negative upper bound is empty. The product groups are the possible grade splits h ; within each split, the component spaces are finite and (3) applies. Counts compose by addition and convolution, while the chosen order of alternatives and grade splits determines ranks.

A list needs a cost for its length if zero-grade elements are available. Assign each element one unit in addition to its own grade. For finite lists of elements of A ,

$$c_{\text{list}(A)}(g) = \sum_{\ell=0}^g \sum_{h_1+\dots+h_\ell=g-\ell} \prod_{i=1}^{\ell} c_A(h_i). \quad (10)$$

The empty inner product contributes one only when $\ell = g = 0$. There are finitely many lengths and grade splits, so every group remains finite. Without the length cost, arbitrarily long lists of a zero-grade element would all occupy grade zero.

For a finite recursive grammar with finite primitive layers, a sufficient condition is a positive total grade cost around every recursive cycle, with nonnegative costs elsewhere. At fixed grade, recursive cycles can then be traversed only finitely many times. Finite branching and acyclic zero-cost steps complete the termination argument. A grammar with a zero-cost recursive cycle needs another well-founded construction; recursive syntax alone provides no proof of finite layers.

These operations agree at the conceptual level with the algebra of finite parts developed by [Duregård, Jansson and Wang \(2012\)](#). The explicit bidirectional contract here additionally requires recovering the chosen constructor, grade split and component values from every admissible object.

4.1 A small expression grammar

For a concrete computing example, take two distinct leaf symbols x, y , a unary negation constructor and an ordered binary addition constructor. Objects are finite syntax trees; node count n includes leaves and constructors. With E_n the number of such trees,

$$E_0 = 0, \quad E_1 = 2, \quad E_n = E_{n-1} + \sum_{h=1}^{n-2} E_h E_{n-1-h} \quad (n \geq 2). \quad (11)$$

Negation contributes the first term. Addition contributes one finite product for each left-child size h . These disjoint constructors give counts 2, 2, 6, 14, 42 at sizes 1 through 5. Order the leaves as x, y , unary trees before binary trees at each size, and binary splits by increasing h , using the left component as least significant. Prefixing the sizes and applying (3) gives both inverses. For example, $x + y$ has size 3, four earlier smaller trees, two earlier unary trees, and binary local rank $0 + 2 \cdot 1 = 2$, hence global rank 8. Reversing those steps recovers its constructor and both leaves.

Under this equality, $x + y$ and $y + x$ are distinct even if their evaluations agree. Ranking mathematical values modulo algebraic identities requires a different canonical domain. The inspected Rust expression codec similarly specifies finite symbol alphabets and constructor distinctions, but bounds depth and arity and materializes its finite catalogue before assigning coordinates and a size-based order [SIM project, 2026a](#). The displayed recurrence is an independent counted construction, not a claim about that codec's execution strategy.

5 Lists: a complete coordinate calculation

5.1 Fixed length and total

Let $W(\ell, s)$ count lists of ℓ nonnegative integers summing to s . For $s \geq 0$,

$$W(\ell, s) = \binom{s + \ell - 1}{\ell - 1} \quad (\ell \geq 1), \quad W(0, s) = \begin{cases} 1 & s = 0, \\ 0 & s > 0. \end{cases} \quad (12)$$

Set $W(\ell, s) = 0$ when $s < 0$. To derive the binomial count, place s units in a row and use $\ell - 1$ separators to divide them into ℓ possibly empty parts. Selecting the separator positions among $s + \ell - 1$ positions determines the list uniquely.

Order these lists by their first entry, then recursively by the remaining entries. For a nonempty $x = (x_1, \dots, x_\ell)$ with total s ,

$$R_{\ell,s}(x) = \sum_{a=0}^{x_1-1} W(\ell - 1, s - a) + R_{\ell-1,s-x_1}(x_2, \dots, x_\ell), \quad (13)$$

with $R_{0,0}() = 0$. The skipped block for first entry a has exactly the number of admissible tails shown in the sum. Unranking selects the unique first-entry block containing the residual coordinate and repeats with that tail's length and total. Length decreases at every step.

For positive integers of length $\ell \geq 1$ and total t , subtracting one from each entry gives $W(\ell, t - \ell) = \binom{t-1}{\ell-1}$ when $t \geq \ell$, and zero otherwise. For bounded entries $0 \leq x_i < b_i$, the unrestricted binomial is generally wrong. The correct count is the coefficient of z^s in

$$\prod_{i=1}^{\ell} (1 + z + \dots + z^{b_i-1}), \quad (14)$$

with an empty factor when $b_i = 0$. Multiplication chooses one admissible value per position; the exponent records their sum. The same bounded tail counts replace W in (13).

Fixed total alone does not make nonnegative lists finite: zero entries may be inserted indefinitely. Positive entries do give finitely many lengths at a fixed positive total. This distinction explains why a length cost or a positivity condition is essential.

5.2 All finite nonnegative lists

Take X to be all finite lists of nonnegative integers, including the empty list. For a list of length ℓ , set

$$g = \ell + \sum_{i=1}^{\ell} x_i. \quad (15)$$

Grade zero consists of the empty list. For $g \geq 1$, order lengths increasingly from 1 to g , then use (13). The length- ℓ block has $\binom{g-1}{\ell-1}$ objects. Hence

$$C_0 = 1, \quad C_g = 2^{g-1}, \quad S_g = 2^{g-1} \quad (g \geq 1). \quad (16)$$

The binomial sum gives C_g ; summing earlier grades, including the empty list, gives S_g . Equivalently, replacing each x_i by $x_i + 1$ produces a positive composition of g . Each of the $g - 1$ gaps between units can be cut or left uncut, giving the same count independently.

The complete rank is

$$R(()) = 0, \quad R(x) = 2^{g-1} + \sum_{k=1}^{\ell-1} \binom{g-1}{k-1} + R_{\ell,g-\ell}(x) \quad (\ell > 0). \quad (17)$$

Every list has finite grade, and the prefix sums are unbounded. The construction therefore covers exactly $\mathbb{N}_0$ and all finite nonnegative lists.

Table 1. The first eight objects under the declared list order.

Rank	List	Grade
0	()	0
1	(0)	1
2	(1)	2
3	(0, 0)	2
4	(2)	3
5	(0, 1)	3
6	(1, 0)	3
7	(0, 0, 0)	3

5.3 From an object to 45, and back

For $x = (2, 0, 1)$, length is 3, total is 3, and grade is 6. Earlier grades contain 32 objects. At grade 6, lengths 1 and 2 contribute $1 + 5 = 6$ objects. Within length 3, a first entry of zero permits four tails, and a first entry of one permits three tails. The tail $(0, 1)$ is first among length-2 tails of total 1. Consequently,

$$R(2, 0, 1) = 32 + (1 + 5) + (4 + 3) + 0 = 45. \quad (18)$$

Conversely, $32 \leq 45 < 64$ identifies grade 6 and leaves residual 13. Skipping the length-1 and length-2 blocks leaves 7 in the length-3 block, whose count is 10. Skipping first-entry blocks of sizes 4 and 3 leaves zero in the block for first entry 2. The remaining length and total are 2 and 1; residual zero selects first entry 0 and final entry 1. Thus unranking 45 recovers $(2, 0, 1)$.

No preceding list was constructed. Only block counts and the selected object's components were needed. The legacy Kotlin nonempty-list convention gives this object rank 44. Adding the empty list at rank zero is an explicit change of coordinate system, not a correction that can silently preserve stored ranks.

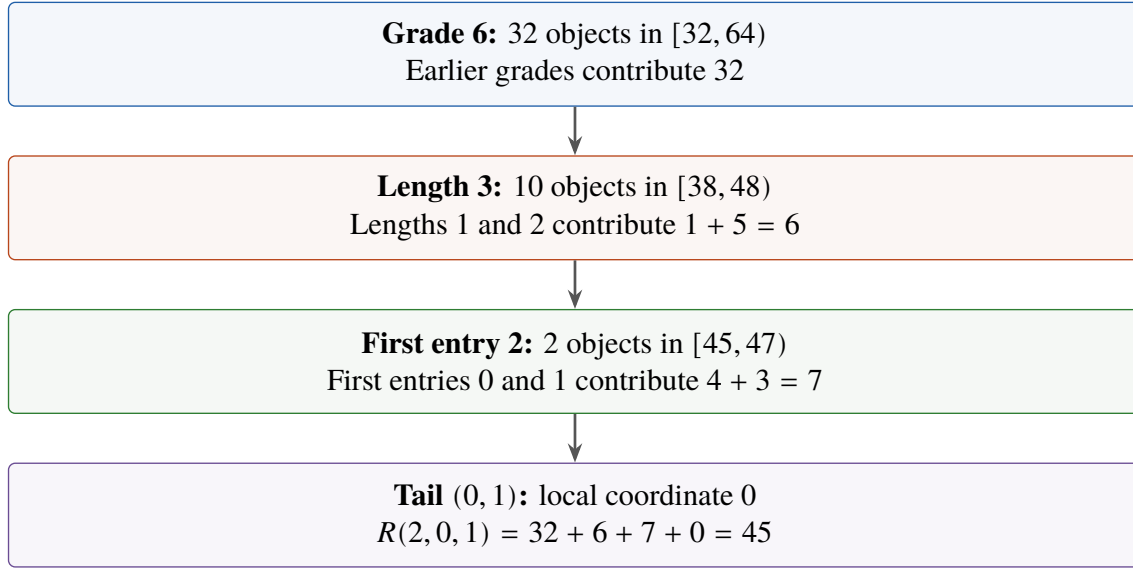


Figure 1. Locating the list (2,0,1) by nested count intervals. The widths in this schematic are not proportional to counts. Every skipped block contributes to the coordinate in equation (18).

6 Permutations and tree shapes

6.1 Permutations of a fixed alphabet

Fix an ordered alphabet of n distinct symbols. At position j , let d_j be the number of still-unused symbols smaller than the selected symbol. Grouping by the next symbol leaves $(n - 1 - j)!$ possible suffixes, so

$$R(\pi) = \sum_{j=0}^{n-1} d_j (n - 1 - j)!, \quad 0 \leq R(\pi) < n!. \quad (19)$$

Repeated factorial blocks determine the inverse choices from the ordered remaining alphabet. The empty permutation has count $0! = 1$ and rank zero. For (2, 0, 3, 1) on $\{0, 1, 2, 3\}$, the digits are (2, 0, 1, 0) and the rank is $2 \cdot 6 + 0 \cdot 2 + 1 \cdot 1 = 13$.

The objects are ordered arrangements. Treating all these arrangements as the same unordered set collapses $n!$ objects into one, making the permutation inverse law impossible under that equality. An insertion-ordered container can preserve traversal order while still using order-insensitive equality; the mathematical contract must resolve that mismatch explicitly.

6.2 Subsets, labeled graphs and coefficient vectors

The same alphabet supports several different object spaces:

Table 2. Equality and order distinguish arrangements from selections.

Object on n ordered symbols	Count	Declared order
Permutation of all symbols	$n!$	Lexicographic arrangements
Subset of cardinality k , $0 \leq k \leq n$	$\binom{n}{k}$	Lexicographic increasing tuples
Arbitrary subset	2^n	Binary membership mask

For $0 \leq a_0 < \dots < a_{k-1} < n$, set $a_{-1} = -1$. The subset's lexical coordinate is

$$R_k(a_0, \dots, a_{k-1}) = \sum_{j=0}^{k-1} \sum_{v=a_{j-1}+1}^{a_j-1} \binom{n-v-1}{k-j-1}. \quad (20)$$

Each summand counts completions after choosing the smaller next element v . The inverse selects the containing block and continues with fewer available elements; the empty subset has rank zero. For arbitrary subsets, the membership bits b_i instead give rank $\sum_{i=0}^{n-1} 2^i b_i$; binary digits recover membership. A fixed-cardinality subset's mask need not equal its dense coordinate among subsets of that cardinality.

A simple undirected graph on fixed labeled vertices $0, \dots, n-1$ selects among $\binom{n}{2}$ possible edges. Order edge positions lexicographically as (i, j) with $i < j$ and use their membership mask, giving $2^{\binom{n}{2}}$ graphs. On three vertices the positions are $(0, 1), (0, 2), (1, 2)$; edges $\{(0, 1), (1, 2)\}$ have bits $(1, 0, 1)$ and rank 5. Binary expansion recovers exactly those edges. This agrees with the inspected graph construction [SIM project, 2026c](#). Relabeling generally changes the object; unlabeled isomorphism classes, connectivity or prescribed degrees need new canonical representatives or accepted completion counts.

A discrete signal example uses a length- ℓ coefficient vector with each entry in $\{0, \dots, b-1\}$, $b \geq 1$: it has b^ℓ objects and the finite-product rank. The inspected signal adapter uses this finite domain and a separate distance $\sum_i |c_i - c'_i|$ [SIM project, 2026c](#). The ordinal alone is not that distance. Interpreting the coordinates as transform coefficients requires specifying the transform and its normalization; these counts do not rank arbitrary real waveforms or order acoustic energy.

6.3 Binary shapes by number of nodes

Consider rooted ordered binary tree shapes with an empty left or right slot permitted. The empty tree has size zero. A nonempty tree consists of a root, a left subtree of size k , and a right subtree of size $n-1-k$. Its counts T_n satisfy

$$T_0 = 1, \quad T_n = \sum_{k=0}^{n-1} T_k T_{n-1-k} = \frac{1}{n+1} \binom{2n}{n}. \quad (21)$$

The decomposition proves the recurrence. To obtain the closed form, the formal power series $T(z) = \sum_{n \geq 0} T_n z^n$ satisfies $T(z) = 1 + zT(z)^2$. The solution with constant term one is $(1 - \sqrt{1-4z})/(2z)$; its formal binomial expansion gives the displayed coefficients. No analytic convergence assumption is required for this use of formal series.

Order the groups by ascending left size and use the first-component-least-significant product convention within a group. For a nonempty shape $t = (L, R)$,

$$R_n(t) = \sum_{j=0}^{k-1} T_j T_{n-1-j} + R_k(L) + T_k R_{n-1-k}(R), \quad k = |L|, \quad (22)$$

with $R_0(\emptyset) = 0$. Selecting the left-size block and dividing its residual by T_k recovers both subtree coordinates. Every recursive subtree has smaller size. The balanced three-node tree has rank 2: the left-size-zero block contains two shapes, and both one-node subtrees have local rank zero. Prefixing the fixed-size counts also ranks the union of all finite sizes.

A full binary tree instead has either zero or exactly two nonempty children at every node. With $\lambda \geq 1$ leaves its count is $T_{\lambda-1}$. Indeed, with $J_1 = 1$, splitting the leaves between two nonempty children gives $J_\lambda = \sum_{a=1}^{\lambda-1} J_a J_{\lambda-a}$, which is precisely the Catalan recurrence after shifting the index. A full binary tree

has $2\lambda - 1$ nodes, as follows by induction over the same split.

Full branching is essential. If unary nodes are admitted, chains of arbitrary length can have one leaf; that leaf-count class is infinite. A validator that checks only the number of leaves cannot justify the finite Catalan count.

For rooted ordered trees with arbitrary finite arity, node count is again a useful grade. The root contributes one and its ordered child list has total subtree size $n - 1$; each nonempty child has positive size, bounding the number of children. The same finite decomposition therefore applies. Ranking shapes still discards labels. Finite label sets can be included as additional product factors; unbounded labels need their own finite grading.

Two further archived shape families illustrate how restrictions change a decomposition without changing its principle:

Table 3. Additional archived families. Their coordinates and constraints are distinct from the Fibonacci family developed next.

Family	Defining restriction	Role in the framework
Golden-mean split trees	Prescribed child leaf counts at each total, with distinct orientations retained	Restrict the allowed split groups
Repeated equal-branch trees	A leaf, or two copies of the preceding tree	One shape at each depth; depth is its coordinate

6.4 Labeled progression trees

Fix a finite rooted ordered topology with N nodes, visited in preorder, and an ordered alphabet of $K > 0$ distinct chord labels. A progression object retains the label at every node, including internal nodes. With label indices $d_0, \dots, d_{N-1}$ and the first index most significant, independent labels give

$$C = K^N, \quad R = \sum_{i=0}^{N-1} d_i K^{N-1-i}. \quad (23)$$

Base- K digits, padded to length N , recover all labels; placing them on the fixed topology gives the inverse. At $K = 1$ there is only one object and rank zero. For a root with two ordered leaves, choose the alphabet (I, V, vi) . The preorder labels (V, vi, I) have digits $(1, 2, 0)$ and rank $1 \cdot 9 + 2 \cdot 3 = 15$ among 27 objects. Dividing 15 successively by 9 and 3 recovers those three digits and their placement. The labels specify chord identities; no voicing, timing or inferred harmonic root is encoded.

Local compatibility can also be counted. Let $A_v(c)$ mean that label c is allowed at node v , and $B_{vw}(c, b)$ that parent label c and child label b are compatible on edge (v, w) . With indicator values zero or one, the number of completions at v conditional on its label is

$$D_v(c) = A_v(c) \prod_{w \text{ child of } v} \left(\sum_b B_{vw}(c, b) D_w(b) \right), \quad C = \sum_c D_{\text{root}}(c). \quad (24)$$

At a leaf the empty product gives $D_v(c) = A_v(c)$. For fixed parent label, children are independent under these local rules, proving the product; alternative labels prove the sum. Rank root labels in alphabet order by their D counts. At each child, skip compatible earlier labels using its D counts and add its conditional subtree coordinate. Combine those child coordinates with first-child-most-significant mixed

radix. Selecting the same blocks in reverse recovers every label; induction from leaves proves both inverse laws. Zero-count choices are skipped. Rules coupling separate branches would need additional state in these completion counts.

As an illustrative restriction, require every child label to differ from its parent, with all labels otherwise allowed. For the three-node topology above, each root label has four completions, so there are 12 objects. With root V , the child alphabet is (I, vi) ; (V, vi, I) now has restricted rank $4 + (1 \cdot 2 + 0) = 6$. Its inverse selects the root- V block, then child digits $(1, 0)$. This is a declared combinatorial rule, not a claim of harmonic quality.

The old Haskell progression construction leaves reverse ranking unimplemented; the Kotlin construction uses unranking to generate trees. The inspected Rust progression catalogue instead fixes a topology and ranks its finite labels [SIM project, 2026b](#). Equation (24) supplies an explicit counted restriction here. A rule derivation, a labeled tree and its flattened musical output are different objects: distinct trees or derivations can produce the same output. These coordinates recover the declared labeled tree, without silently identifying those outputs.

7 Fibonacci trees, limerick contours and the next level

7.1 A restricted tree family with an exact count

The archived Haskell and Mathematica sources contain a more selective family than all binary shapes: *Fibonacci trees*. Write $\mathcal{F}_1$ for the single leaf and $\mathcal{F}_2$ for the two-leaf fork. For $n \geq 3$, a tree at level n has children from levels $n - 2$ and $n - 1$, in either order. Thus

$$\mathcal{F}_n = (\mathcal{F}_{n-2} \times \mathcal{F}_{n-1}) \sqcup (\mathcal{F}_{n-1} \times \mathcal{F}_{n-2}), \quad Q_n = 2Q_{n-1}Q_{n-2}, \quad Q_1 = Q_2 = 1. \quad (25)$$

Let $F_0 = 0, F_1 = 1, F_{n+1} = F_n + F_{n-1}$. Each level- n tree has F_{n+1} leaves, so the two child orders in (25) are disjoint. Induction on the count recurrence gives $Q_n = 2^{F_n-1}$: the exponents satisfy $e_n = 1 + e_{n-1} + e_{n-2}$ with $e_1 = e_2 = 0$.

Table 4. Fibonacci leaf counts and the sizes of their ranked shape families.

Level n	1	2	3	4	5	6	7
Leaves F_{n+1}	1	2	3	5	8	13	21
Shapes Q_n	1	1	2	4	16	128	4096

For this example use the recovered Haskell convention: the smaller-level child comes first in group zero, and the left child is the most significant component. For group $b \in \{0, 1\}$ with child levels (a, c) , its rank is

$$R_n(L, R) = bQ_{n-2}Q_{n-1} + Q_cR_a(L) + R_c(R). \quad (26)$$

Unranking separates the orientation block, then the left and right ranks by quotient and remainder. Both child levels decrease. Levels 1 and 2 each have rank zero. The duplicate leaf-level convention in the old Haskell source (levels 0 and 1) is normalized here by starting the family at level 1.

The underlying unordered shape is the leaf-Fibonacci tree f_{n-1} of [Dossou-Olory \(2018\)](#). That work counts nonisomorphic subtrees induced by selected leaves. Here left/right order is part of the object, and Q_n counts the ordered realizations of the complete recursive shape. These are different counting questions; the shared tree recurrence is established context for the rhythmic construction.

7.2 Reading a tree as a rhythm

An archived musical construction assigns duration by depth. Put the root at depth zero, read leaves from left to right, and give a leaf at depth d_i duration

$$\tau_i = 2^{-d_i}, \quad \sum_i \tau_i = 1. \quad (27)$$

The sum is one because each binary branch divides its parent's duration into two equal parts. Reading leaves in order places these parts consecutively. For the full binary trees here, the exact duration sequence determines the tree: the root boundary is the unique cumulative sum equal to $1/2$, and doubling the durations on each side repeats the reconstruction. Consequently the tree rank also distinguishes these exact normalized rhythmic patterns. This conclusion uses their ordered, aligned binary subdivisions, not arbitrary lists of positive durations.

7.3 Exactly which duration sequences are admitted?

The duration map gives a membership test as well as an encoding. This matters when a musician supplies a pattern and asks whether it belongs to the ranked family.

Theorem 2 (Fibonacci rhythm recognition). *Let τ be an ordered positive duration vector of total one and let $n \geq 1$. It is the duration vector of a tree in $\mathcal{F}_n$ exactly when the following recursive conditions hold. At level 1 it is (1); at level 2 it is (1/2, 1/2). At level $n \geq 3$ it has F_{n+1} entries and a proper prefix of total 1/2, containing either F_{n-1} or F_n entries. In the first case, doubling the durations in the prefix and suffix must give admitted vectors at levels $n-2$ and $n-1$; in the second case the levels are reversed. The admitted tree and its rank are unique.*

Proof. A tree's two root branches each occupy half the total duration and have the stated leaf counts. Doubling removes this first subdivision, so necessity follows recursively. Conversely, the two recursively admitted vectors give trees at the required child levels; joining them gives a tree of $\mathcal{F}_n$ with the supplied durations. Positivity makes the half-total prefix unique, and the two child leaf counts differ. Thus the orientation and both subtrees are uniquely determined. The levels decrease to the base cases, proving termination and uniqueness by induction. Equation (26) then recovers the unique coordinate. $\square$

Dyadic durations summing to one are insufficient: (1/4, 1/2, 1/4) has no half-total prefix. Even an aligned binary rhythm with the right leaf count can fail: (1/2, 1/4, 1/8, 1/16, 1/16) has five leaves, but its root split is $1 + 4$ rather than the required $2 + 3$. The criterion therefore distinguishes the Fibonacci family from both arbitrary dyadic lists and all binary subdivisions.

7.4 The five-part limerick contour

Let E denote a leaf and $C = (E, E)$ the two-leaf fork. The five-leaf tree

$$T = (C, (C, E)) \in \mathcal{F}_4, \quad R_4(T) = 1, \quad (\tau_i) = \frac{1}{8}(2, 2, 1, 1, 2) \quad (28)$$

has the contour **long, long, short, short, long**. In the Mathematica ordering the same tree has one-based rank 3. These are reconstructed coordinates of the same shape under the two archived orders.

That contour supplies the recognizable five-part outline of a limerick. A metrical realization assigning three feet to each long part and two to each short part gives (3, 3, 2, 2, 3), the standard line pattern described by [Poetry at Harvard \(n.d.\)](#). The two realizations have different units: (28) gives binary time

proportions of 2 : 1, while the poetic assignment gives line lengths of 3 : 2. Internal anapestic stress and the AABBA rhyme scheme are further choices in realizing a poem. The archive establishes the tree family and the depth-to-duration rule; this particular limerick identification is a reconstruction prompted by the author’s recollection.

7.5 The eight-leaf “super-limerick” family

The next level has eight leaves and sixteen ranked patterns. It combines a three-leaf tree with a five-leaf tree, with two orientations and independently ranked children. One explicit continuation keeps T intact as the right child and takes $B = (C, E)$ on the left:

$$S = (B, T) \in \mathcal{F}_5, \quad R_5(S) = 1 \cdot 4 + 1 = 5, \quad (\tau_i) = \frac{1}{16}(2, 2, 4, 2, 2, 1, 1, 2). \quad (29)$$

Its last five durations reproduce the limerick contour at half the total duration of (28). The first three add a complementary phrase. Figure 2 shows the hierarchy and the resulting timing.

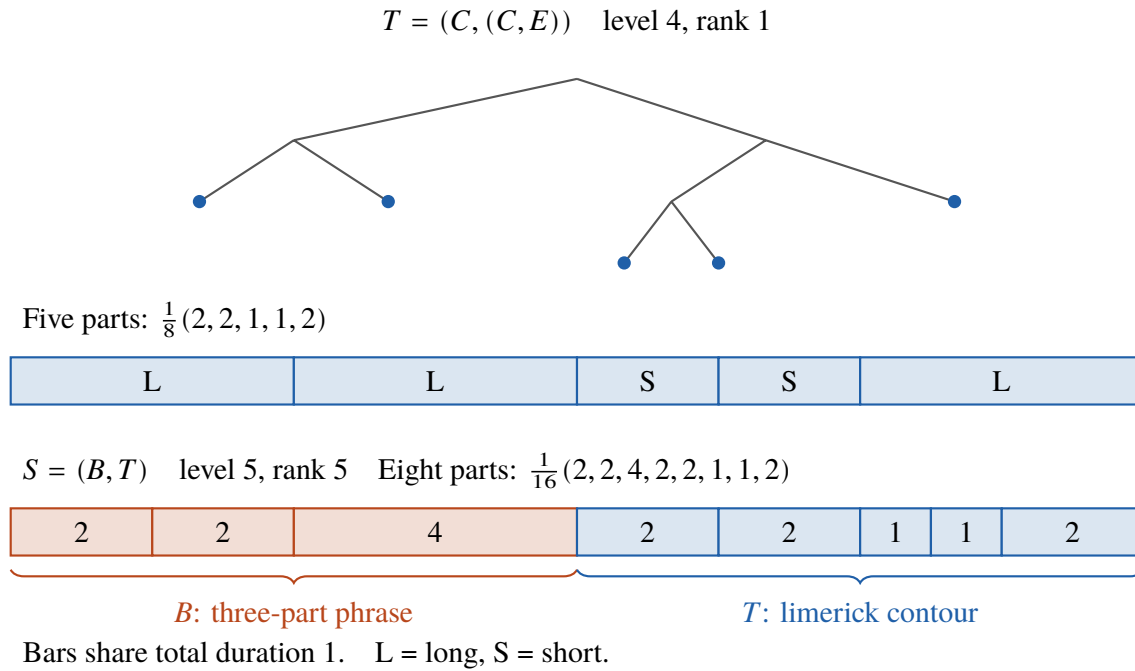


Figure 2. A five-part limerick contour and one eight-part continuation. Horizontal bars show exact normalized durations; vertical strokes mark event onsets and the right endpoint. The tree schematic shows the five-leaf shape T ; the continuation S joins B and T . The poetic three-foot/two-foot interpretation is separate from these binary durations.

Here “super-limerick” names the next structural family, with (29) as one selected member. The recurrence gives sixteen choices, not one compulsory continuation. At the following levels the same construction gives 128 thirteen-leaf patterns and 4,096 twenty-one-leaf patterns. Ranking makes each candidate addressable and exactly recoverable. Choosing phrase boundaries, stresses, rhyme or a preferred performance gives an additional musical or poetic problem on that finite space.

The starting boundary is part of the rhythmic object. For example, rotating (28) to $(1, 2, 2, 2, 1)/8$ removes the half-total prefix and leaves the family. Cyclic rhythm classes would need their own domain and canonical representatives. The present coordinates recover an anchored phrase exactly.

7.6 Counted extensions that preserve a motif

Fix one ordered motif $M \in \mathcal{F}_k$, with $k \geq 2$. A tree *contains* M when some node, with all its descendants, is exactly M . In duration terms this retains the entire motif inside an aligned subinterval, scaled to that interval's length. It is a precise structural condition; arbitrary fragments across branch boundaries are outside it.

Let A_n count trees avoiding M , and H_n those containing it. Then

$$A_n = Q_n \ (n < k), \quad A_k = Q_k - 1, \quad A_n = 2A_{n-2}A_{n-1} \ (n > k), \quad H_n = Q_n - A_n. \quad (30)$$

At lower levels the motif is too large to occur. At level k , only M itself contains M . Above level k , the root cannot equal M , and avoidance means avoidance in both children. The two orientations and independent child choices prove the recurrence. For the fork motif, every higher tree contains it and the recurrence gives zero avoidance. The leaf motif can be handled separately: every tree contains it, so $A_n = 0$ at every level.

For the chosen limerick tree T , with $k = 4$, the counts are:

Table 5. Exact counts for retaining the complete ordered limerick motif.

Level	Leaves	All patterns Q_n	Avoid T : A_n	Retain T : H_n
4	5	4	3	1
5	8	16	12	4
6	13	128	72	56
7	21	4,096	1,728	2,368

Counting also gives direct dense coordinates in the selected family. At each level above k , first choose the root orientation in the order of (26). For child levels (a, c) , classify a child by flag 0 (avoids the motif) or 1 (contains it). Within a containing orientation block, order the flag pairs as $(0, 1)$, $(1, 0)$, $(1, 1)$, with respective counts

$$A_a H_c, \quad H_a A_c, \quad H_a H_c. \quad (31)$$

The avoiding block uses only $(0, 0)$ and has count $A_a A_c$. Skip zero-sized blocks. Within each block use the left component as most significant and the recursively selected child coordinates. Below k the avoiding rank is the ordinary rank. At k the containing space is the singleton M ; the avoiding space deletes M 's ordinary coordinate and closes the gap.

These cases are disjoint and exhaustive, so the compositional inverse theorem proves both inverse identities. Ranking sums preceding block counts; unranking selects a block by its count and splits the residual into its two child ranks. No rejected tree needs to be constructed. At level 5 the four conditional coordinates 0, 1, 2, 3 correspond to ordinary ranks 1, 5, 10, 11. In particular, the continuation S in (29) has conditional rank 1. This conditional block order is explicit; at higher levels it need not be the subsequence order inherited from the ordinary rank.

The count can be refined to the number of motif occurrences. For $M = T$, put $P_n(z) = \sum_{t \in \mathcal{F}_n} z^{o_T(t)}$, where $o_T(t)$ counts nodes whose complete subtree is T . Then

$$P_n(z) = Q_n \ (n < 4), \quad P_4(z) = 3 + z, \quad P_n(z) = 2P_{n-2}(z)P_{n-1}(z) \ (n > 4). \quad (32)$$

Above level 4 the root is not an occurrence, and the two child occurrence counts add, proving the product rule. For example, $P_6(z) = 72 + 48z + 8z^2$: of the 56 containing thirteen-part patterns, 48 contain one copy and eight contain two. The constant term recovers A_n . Fixing an occurrence count instead of a Boolean flag gives another counted restriction, with product blocks split by the two child occurrence counts.

This gives a concrete use of ranking in composition: retain a chosen phrase, select a family size or repetition count, and recover any admitted extension directly. It does not claim that the resulting patterns exhaust useful rhythms. Jacquemard, Ycart and Sakai (2017) develop broader rhythm-tree languages and enumerate equivalent notations of given durations by grammar weights. Here the objects are distinct exact duration vectors in a restricted binary family, with motif-conditioned dense rank and unrank. The recognition and counting results specify that narrower exploration space completely.

8 Ranking melodies by total pitch movement

8.1 A musical quantity before a coordinate

The archived melody constructions already suggest a useful representation: successive pitch differences. The Haskell pitch-list construction reconstructs them from starting pitch zero. The Mathematica melody construction combines pitch differences with successive duration ratios; its default reconstruction starts at pitch zero and first duration one. These transformations encode transposition and duration-scale normalization. A claim to recover arbitrary absolute pitches and times would additionally require their starting values. The rhythm convention used below instead fixes total duration to one. Successive duration ratios determine a positive vector up to scale, so either normalization can be chosen, but their numerical duration coordinates differ.

For an explicit musical order, fix $m \geq 2$ monophonic note events, a positive normalized duration vector, and a starting pitch $p_0 = 0$ relative to a chosen reference. Let $d = m - 1$ and $p_i \in \mathbb{Z}$ be semitone positions. Equal temperament assigns frequency $f_i = f_{\text{ref}} 2^{p_i/12}$; pitch here is logarithmic, so a semitone has the same coordinate size in every register. The ordered pitches, with their fixed event boundaries, define equality. Repeated pitches remain distinct note events. Set

$$\Delta_i = p_i - p_{i-1}, \quad p_i = \sum_{j=1}^i \Delta_j, \quad V(p) = \sum_{i=1}^d |\Delta_i|. \quad (33)$$

Thus V is total pitch travel in semitones. It measures how far the melody moves through pitch space; at fixed d , its mean absolute interval is V/d . The zero-motion melody comes first, then melodies with one semitone of total motion, then two, and so on. Within one motion value, order the signed interval vectors lexicographically, with ordinary ascending integer order.

Transporting signed integers to nonnegative integers is enough for a bijection, but need not respect this meaning. The archived entrywise order $0, -1, 1, -2, 2, \dots$ assigns indices $0, 1, 2, 3, 4, \dots$. The interval vector $(-1, -1, -1)$ has encoded total 3 and pitch travel 3, whereas $(0, 0, 2)$ has encoded total 4 and pitch travel 2. Increasing that encoded grade can decrease actual movement. Grouping by V makes the desired measurement primary.

8.2 Finite motion groups and both inverse maps

Let $N_d(q)$ count d signed intervals of total absolute size q , setting counts with $q < 0$ to zero. The exact counts are

$$N_0(q) = \mathbf{1}_{q=0}, \quad N_d(q) = \sum_{a=-q}^q N_{d-1}(q - |a|) \quad (d \geq 1), \quad (34)$$

$$N_d(0) = 1, \quad N_d(q) = \sum_{j=1}^{\min(d,q)} \binom{d}{j} 2^j \binom{q-1}{j-1} \quad (q > 0). \quad (35)$$

The recurrence groups by the first interval. For the closed form, choose the j nonzero positions, their 2^j signs, and a positive composition of q into their magnitudes. These choices are unique and exhaustive. Every group is finite, and for $d \geq 1$ every $q \geq 0$ has at least one member.

Write $S_d(q) = \sum_{h=0}^{q-1} N_d(h)$. For a valid interval vector of motion q , define its local coordinate recursively by

$$L_{d,q}(\Delta) = \sum_{a=-q}^{\Delta_1-1} N_{d-1}(q - |a|) + L_{d-1,q-|\Delta_1|}(\Delta_2, \dots, \Delta_d), \quad (36)$$

$$R_{\text{pitch}}(p) = S_d(V(p)) + L_{d,V(p)}(\Delta), \quad L_{0,0}(\Delta) = 0. \quad (37)$$

To invert a nonnegative rank, find the unique motion band $S_d(q) \leq r < S_d(q) + N_d(q)$. From its residual, select the first interval by the successive counts in (34); subtract the preceding counts and repeat with one fewer interval and the remaining motion budget. Finally integrate the recovered intervals using (33).

Theorem 3 (Melody coordinate and motion order). *For the fixed event count, rhythm and starting pitch above, (37) is a bijection from the integer-pitch melodies to $\mathbb{N}_0$. Its inverse is the band and interval reconstruction just described. A larger coordinate never has smaller total pitch travel, and every increase in total pitch travel gives a larger coordinate.*

Proof. Anchored pitch sequences and their interval vectors are inverse descriptions. The finite motion groups partition all such vectors; first-interval blocks partition each group and descend in d . The compositional inverse theorem therefore supplies both inverse identities. Nonempty bands at every q make their prefix sums unbounded, so every nonnegative coordinate is reached after finitely many bands. Ordering those bands proves (7) with $g = V$. $\square$

This is a weak monotonicity guarantee: the many alternatives in one band have equal V . The coordinate distinguishes them without pretending that the tie order adds musical magnitude. A useful display gives both V and R .

8.3 A complete three-note example

Choose rhythm $(1/4, 1/4, 1/2)$ and pitches $(0, 2, 0)$, for example C4–D4–C4. The interval vector is $(2, -2)$ and $V = 4$. For two intervals, $N_2(0) = 1$ and $N_2(q) = 4q$ for $q > 0$. Hence

$$S_2(4) = 1 + 4 + 8 + 12 = 25, \quad L_{2,4}(2, -2) = 1 + 2 + 2 + 2 + 2 + 2 = 11, \quad R_{\text{pitch}} = 36. \quad (38)$$

The six skipped first-interval blocks have first intervals $-4, -3, -2, -1, 0, 1$. At first interval 2 the remaining motion is 2; the tail -2 is its first choice. Conversely, rank 36 lies in the band $[25, 41)$ and

leaves residual 11. Skipping those six blocks leaves zero in the first-interval-2 block, selecting tail -2 . Integrating recovers $(0, 2, 0)$.

Table 6. The same outward-and-returning contour with increasingly large excursions.

Pitch offsets	Total travel V	Coordinate R_{pitch}
$(0, 0, 0)$	0	0
$(0, 1, 0)$	2	10
$(0, 2, 0)$	4	36
$(0, 4, 0)$	8	136
$(0, 7, 0)$	14	406

Figure 3 shows the first four examples on a common pitch and time scale.

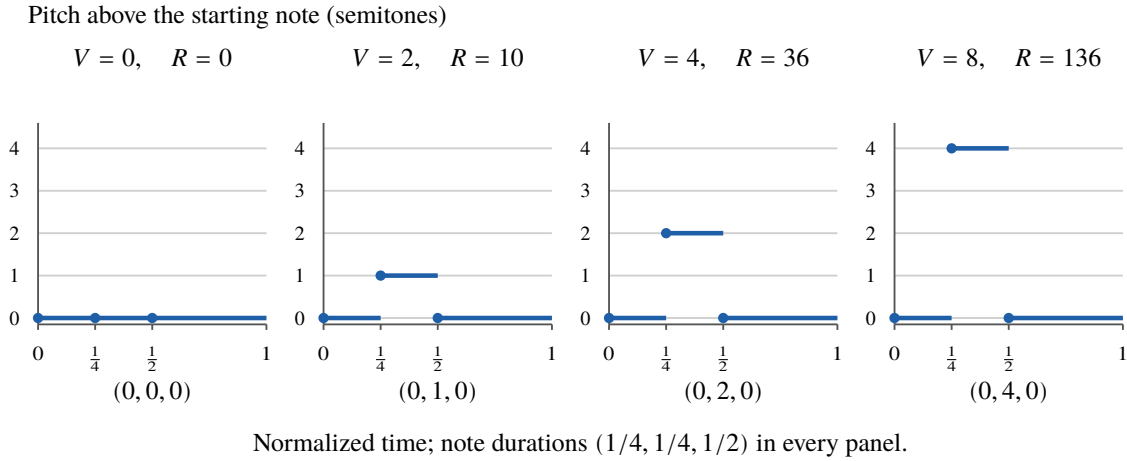


Figure 3. Increasing pitch excursions at fixed rhythm, starting pitch and event count. Horizontal segments are held notes; dots mark their onsets. All four panels use the same semitone scale. V gives total pitch travel and R the exact coordinate.

8.4 Adding rhythms and an instrument's range

A complete pitch-and-duration melody can use a finite ranked rhythm family $\mathcal{R}$ of size $M > 0$, all with the same m events and normalized total duration. For rhythm $\rho \in \mathcal{R}$, define

$$R_{\text{melody}}(\rho, p) = MR_{\text{pitch}}(p) + R_{\mathcal{R}}(\rho). \quad (39)$$

Quotient and remainder modulo M recover the pitch and rhythm coordinates. Both component inverse laws give both melody inverse laws. Each motion band has $MN_d(q)$ objects, preserving the monotonicity in V . This factorization assumes the same M rhythms are independently available for every pitch sequence. Pitch-dependent timing restrictions need their own joint completion counts. For example, $\mathcal{R}$ can be the four five-part Fibonacci rhythms, or the four eight-part rhythms that retain T . Selecting the latter preserves a rhythmic motif while increasing pitch travel across bands. No monotonicity in rhythmic complexity is asserted within a band.

For a finite instrument, choose a finite ordered pitch set P containing the fixed starting pitch. Let $D_j(p, q)$ count continuations of j further notes from current pitch $p \in P$ with remaining travel q . Then

$$D_0(p, q) = \mathbf{1}_{q=0}, \quad D_j(p, q) = \sum_{v \in P} D_{j-1}(v, q - |v - p|), \quad (40)$$

with zero for negative budgets. Group by increasing q , then by ascending next pitch v using these completion counts. The same prefix subtraction and induction construct both inverse maps. There are $|P|^d$ melodies and no travel above $d(\max P - \min P)$. A singleton pitch set has only the stationary melody. This incorporates range into the counted domain; clipping or octave-folding unbounded results afterward would merge distinct objects.

8.5 What the musical interpretation establishes

Total travel is an intelligible meaning of “larger”: more accumulated pitch movement over the same number of notes. It does not measure register, pitch range, tonal tension or quality. For example, $(0, 3, 0, 3, 0)$ travels 12 semitones within a range of 3, whereas $(0, 2, 4, 6, 8)$ travels only 8 within a range of 8. The first receives the larger motion rank, although the second spans more pitch space. Reflection of a melody preserves V ; the signed lexicographic tie order distinguishes it without assigning a preference. Nor does total travel determine the largest leap: intervals $(3, 3)$ have travel 6 and largest leap 3, whereas $(5, 0)$ have travel 5 and largest leap 5. Greater travel gives a greater mean absolute interval at fixed note count, without guaranteeing a greater maximum interval.

Pitch proximity has a documented role in models of melodic perception, alongside range and tonal context [Temperley, 2008](#). That supports considering interval size as a meaningful variable; it does not validate total travel as a universal perceptual scale. A listening study could compare pairs at fixed event count, rhythm, reference pitch, timbre, tempo and loudness, ask which has more pitch movement, and test whether judgments increase with V . Matching range and direction patterns where possible would distinguish this quantity from neighboring explanations. No such listening experiment is reported here.

The mathematical result is already useful for controlled exploration: a coordinate recovers every declared note and duration, a motion band bounds the search, and its ordering has a stated guarantee. Perceptual validation can then address that particular guarantee’s musical usefulness.

8.6 Existing bounded musical orders

The inspected Rust musical codecs already separate a canonical coordinate from musically motivated orders. They construct bounded catalogues of durations, notes, rests, articulations or chord sequences, assign coordinates by a lexical key, and sort those coordinates for exploration [SIM project, 2026a](#). This is a finite materialization strategy; the completion-count construction above avoids requiring that entire catalogue.

For extracted pitch classes $c_0, \dots, c_{s-1} \in \{0, \dots, 11\}$, the Rust low-motion score is

$$V_{\text{pc}} = \sum_{i=1}^{s-1} \min(|c_i - c_{i-1}|, 12 - |c_i - c_{i-1}|). \quad (41)$$

Rests are omitted before forming this sequence, so motion is measured between successive pitched events, including those separated by rests. With fewer than two pitched events it is zero. The alternate order sorts by this score, then the span $\max c_i - \min c_i$ (zero for no pitches), then the lexical key. That span uses the chosen pitch-class representatives, not absolute register. By contrast, (33) retains octave displacement:

a leap of 12 semitones has $V = 12$ but circular pitch-class distance zero. The article's ties use signed interval tuples. The two constructions order different declared objects and do not assign interchangeable coordinates.

The duration-simplicity order sorts first by event count, then by sums of reduced duration denominators and numerators, then by its lexical key. Progression orders use circular motion of a selected pitch from each chord; the helper selects the first stored pitch, rather than inferring a harmonic root. Its cadence rule prioritizes endings 7, 0, then other endings at 0, then the rest, with motion and lexical ties. These are explicit exploration rules on a bounded catalogue. They establish neither a universal simplicity scale nor a general theory of cadence. The counted musical constructions in this article make their domains, measurements and access mechanism explicit while preserving that useful separation of identity from exploration order.

9 Transport, canonical forms and filtered spaces

If $f : X \rightarrow Y$ is a specified bijection and Y is ranked, coordinates transport along it:

$$R_X = R_Y \circ f, \quad U_X = f^{-1} \circ U_Y. \quad (42)$$

For example, the integer-to-natural-number bijection

$$f(z) = \begin{cases} 2z & z \geq 0, \\ -2z - 1 & z < 0 \end{cases} \quad (43)$$

orders integers as 0, -1, 1, -2, 2, Transporting each entry lets the list construction rank finite lists of arbitrary integers.

A computable sequence of distinct values $s_0, s_1, \dots$ similarly gives $U(r) = s_r$ on its declared range. For example, with the increasing prime sequence 2, 3, 5, 7, ..., the value 7 has rank 3. Ranking can search for the value; strict increasing order permits rejection after passing a nonmember. The order and computability assumptions matter, and sequential search gives no fast-inversion guarantee. Repeated sequence values cannot have a unique value rank without deduplication: otherwise the objects must include the occurrence index. This states the contract needed by the archived sequence wrappers, rather than inferring a bijection from indexing alone.

The compatibility condition in (42) concerns actual domains and images. Combining two nominal rank operations and taking the smaller of their counts does not establish it. For example, $x \mapsto x + 1$ on $\mathbb{N}_0$ is an injective encoding into positive integers, but it leaves coordinate zero unused and its proposed inverse sends zero outside the domain. The same shift is perfectly useful as a positive *cost* in (15). A cost need not itself be a dense rank.

Canonicalization addresses a different issue: multiple descriptions of one object. Positive rational numbers have unique representations $\prod_i p_i^{e_i}$ using the ordered primes and an integer exponent vector of finite support. Use the empty vector for 1 and otherwise end the vector at its last nonzero exponent. Unique factorization proves uniqueness; arbitrary trailing zeros would destroy it. Transport (43) entrywise and grade the resulting finite lists. Each fixed-grade canonical subset is finite, but its count and selection rule must enforce the final-nonzero condition. Ranking all raw vectors and then trimming them is many-to-one and loses invertibility. This gives a valid construction route, not a claim that its counting cost is small.

9.1 Exact ratios: canonical values before coordinates

A direct alternative ranks positive rational values through their unique reduced pairs (a, b) , where $a, b \geq 1$ and $\gcd(a, b) = 1$. Give the pair arithmetic height $h = \max(a, b)$ and order each height lexicographically by (a, b) . Write $\varphi(h)$ for the number of integers in $\{1, \dots, h\}$ coprime to h . Then

$$C_1 = 1, \quad C_h = 2\varphi(h) \ (h > 1), \quad S_h = \sum_{j=1}^{h-1} C_j. \quad (44)$$

At height $h > 1$, a reduced pair lies on exactly one of the sides $a = h$ or $b = h$. Each side has $\varphi(h)$ members; their corner is excluded by coprimality. Height 1 consists only of $(1, 1)$.

For $1 \leq a \leq h$, let $B_h(a)$ count admissible denominators. At $h > 1$, it is $\mathbf{1}_{\gcd(a, h)=1}$ when $a < h$, and $\varphi(h)$ when $a = h$; also $B_1(1) = 1$. The dense rank of a reduced value is

$$R_{\mathbb{Q}}(a/b) = S_h + \sum_{u=1}^{a-1} B_h(u) + \#\{v : 1 \leq v < b, \max(a, v) = h, \gcd(a, v) = 1\}. \quad (45)$$

To invert, select the height interval, then the numerator block using B_h , then the residual-th admissible denominator in increasing order. The unique reduced pair proves object recovery; disjoint exhaustive finite blocks prove coordinate recovery. Every height is nonempty, so every nonnegative rank is reached. The first seven values are 1, $1/2$, 2, $1/3$, $2/3$, 3, $3/2$. Thus $3/2$ has rank $3+2+1 = 6$: three objects at earlier heights, two earlier numerator blocks at height 3, and denominator 1 before denominator 2. Unranking 6 reverses those choices.

This order measures reduced arithmetic height, not numerical ratio magnitude or consonance. If octave equivalence is wanted, choose the unique representative in $[1, 2)$ under $q \sim 2^k q$ and count only reduced pairs satisfying $b \leq a < 2b$. The previous height counts must then be restricted too; folding every decoded value afterward merges coordinates. For example, $3/2$ and $3/4$ are distinct positive ratios but the same octave class.

This distinction affects the inspected bounded Rust prime-vector construction: its decoder reconstructs a ratio and then applies the selected octave policy [SIM project, 2026b](#). With octave reduction enabled, exponent vectors for 1 and 2 both decode to 1. Thus all raw exponent-vector coordinates cannot be a bijection onto octave classes. Disabling that quotient preserves the mathematical vector distinction, but finite numerator and denominator storage can still overflow for some vectors in the nominal exponent range. The exact construction above uses mathematical integers and a declared canonical domain; it does not certify every implementation policy or numeric range.

9.2 Filtering candidates and identifying outputs

More generally, suppose a predicate P selects objects from a ranked space. For an accepted coordinate r , its dense coordinate in the selected space is

$$R_P(U(r)) = \sum_{j=0}^{r-1} \mathbf{1}_{P(U(j))}. \quad (46)$$

Its inverse selects the corresponding accepted coordinate of the original space. Merely rejecting objects preserves gaps in the old coordinates. Efficient filtering therefore requires counts of accepted objects or another selection construction. For a decidable predicate, scanning terminates for an existing selected

coordinate, but need not decide that a requested further coordinate does not exist. Sets and maps similarly need a canonical order and uniqueness constraints; unconstrained list counts do not establish their counts. The archived chord experiments provide a concrete use of ranked candidates: decode integer lists, interpret them as ratios, sort and remove duplicates, apply pitch-class and octave transformations, then retain selected chords. Those steps can merge several candidates into the same output. A candidate’s coordinate remains useful for replaying the experiment, but it is not automatically a dense coordinate among unique accepted chords. That further space needs an equality choice—such as exact voicing, a pitch-class set or a transposition class—and counts over its canonical objects. Predicate filtering alone closes gaps between accepted candidates; it does not remove duplicate outputs introduced by a many-to-one transformation.

10 What changes across implementations?

The inspected versions support a common construction, but they are not one interchangeable integer format. The following table records differences that affect the mathematics rather than programming syntax.

Table 7. A shared method does not imply shared numeric coordinates.

Aspect	Inspected convention and consequence
Index origin	The Mathematica layer uses one-based ranks; the other compared cores use zero-based ranks.
Product order	The Java and Kotlin group cores use the first component as least significant. The Haskell base-list construction and Rust grouped products use it as most significant.
List grouping	The Mathematica positive-total list family orders lengths downwards within a total. The worked list construction orders them upwards.
Empty list	The Kotlin variable-list family enumerates nonempty lists. Equation (17) explicitly includes the empty list.
Tree objects	Shape constructors discard payload labels; full binary leaf counts require an arity restriction.
Musical quantities	Bounded Rust melody orders use circular pitch classes; the article’s travel order retains absolute semitone displacement.
Access strategy	The inspected Rust music and expression codecs materialize bounded catalogues; the displayed recurrences count completions.
Quotients	Reducing ratios or folding octaves changes the object domain and requires canonical counts.
Execution domain	A mathematical unbounded coordinate can encounter bounded grades or an exhausted computation budget.

For reproducible coordinates, record the space definition, equality, normalization, group order, within-group order and version. A stored integer without that information is incomplete. Converting between two verified coordinate systems for the same space means decoding with the old system and encoding with the new one; a constant offset works only for a specifically proved change such as inserting the empty list before every nonempty list.

The current public Rust source makes grades, counting limits and separate ordering objects explicit

[SIM project, 2026a](#). Source inspection also shows why an interface cannot substitute for the preceding proofs: its generic grouped codec reports no total count even for a finite grammar, and a predicate guard delegates counting to its inner grammar. Set and map grammar forms are not supported by that grouped codec. The algebra in this article specifies what these cases require; it does not certify every declared grammar form as an implemented bijection.

11 What a structural coordinate makes possible

11.1 Direct access and reproducible work

Counting entire blocks avoids constructing their individual members. A saved coordinate can reproduce an object, and a finite interval can be partitioned into disjoint subintervals for independent workers. Completeness then has a precise scope: every coordinate in the declared interval, under the same space and version. Infinite enumeration is *fair* in the modest sense that every object at a finite coordinate is eventually reached, provided each step terminates. Fairness is not a useful time bound for distant objects.

There is no universal constant-time or logarithmic-time consequence. A linear group search through grade g uses up to $g + 1$ count queries before descending into the chosen group. Counts may themselves be expensive. Memoizing them avoids repeated subproblems, but consumes space. For example, the Catalan recurrence through size n takes $O(n^2)$ additions and multiplications of integers with an $O(n)$ -entry table. This counts arithmetic operations, not bit operations: the integers grow. Decoding also has to construct the output, and scanning subtree split counts adds work. These are mathematical cost statements, not measured performance claims about the implementations.

11.2 Sampling and search are extra choices

For a finite space of size $C > 0$, a uniform coordinate in I_C produces a uniform object by bijectivity. Choosing groups uniformly and then objects uniformly within each group generally does not: a member of group g receives probability proportional to $1/C_g$. Uniform sampling over all objects requires choosing that group with probability C_g/C .

There is no uniform probability distribution assigning the same probability to every object of a countably infinite space. A positive common probability has infinite total mass, while a zero common probability has total zero. Infinite spaces therefore require an explicit finite cutoff or a nonuniform law, for example a distribution on grades followed by uniform selection within a nonempty grade.

A coordinate alone supplies neither a preference score nor a semantic distance. Under (17), the g -entry zero list has rank $2^g - 1$ and the singleton (g) has rank 2^g . Their coordinates are adjacent, but their unit-cost sequence edit distance is g : $g - 1$ deletions and one substitution are necessary and sufficient. Adjacency supplies no bounded structural similarity.

The melody construction gives a deliberate additional guarantee: its bands are ordered by V . Even there, coordinate differences do not measure pitch distance, and the lexicographic alternatives within a band need not be close. Choosing a meaningful grade solves one specified ordering problem.

Variation can operate on the object and then return to its coordinate. At the fixed rhythm used in (38), editing $(0, 2, 0)$ to $(0, 1, 0)$ changes its coordinate from 36 to 10. This is one note edit, not a distance of 26 in musical space. For a bounded instrument or constrained rhythm family, the edited object must still satisfy that space's admission conditions before being re-ranked. The inspected Rust neighborhood machinery likewise generates edited candidates and validates their coordinates. Such operations support exploration without making rank itself a neighborhood, objective or guarantee of an optimal result.

On a finite space, an explicitly invertible permutation can separate search order from canonical identity. A score with ties needs a declared tie-breaker to define such an order. On an infinite space, a score ordering may give some objects infinitely many predecessors or have no first object. A scoring rule alone consequently supplies neither a dense ranking nor a guarantee of finding an optimum. The Rust separation between canonical ordinals and orderings is useful in precisely this sense [SIM project, 2026a](#).

12 Verification and the limits of round trips

Mathematical proofs establish the stated constructions; finite checks connect those constructions to executable behavior. An isolated compilation of the archived Kotlin rank sources and their original list, tree and arithmetic dependencies used Kotlin/JVM 2.4.10 and JRE 21.0.7. Focused probes exercised 714 fixed-length, fixed-total lists, 626 node-count tree shapes, 197 full leaf-count shapes, 873 permutations and 256 variable-list coordinates. These probes checked counts and inverse behavior on the exercised cases; shape and ordered-sequence comparisons were used where host-container equality did not express the intended object equality. This was not a full historical build or a performance benchmark.

Four small counterexamples clarified the specification. A shifted composition accepted coordinate zero before failing to decode it. A variable-list predicate admitted the empty list although its rank operation rejected that list. A unary one-leaf tree passed a leaf-count check but decoded as a different shape. Two different permutation orders compared equal as sets. The mathematical remedies are respectively compatible images, consistent empty-object domains, full-tree validation, and ordered equality. They are conditions of the article’s model; the archived implementations were preserved unchanged.

An independent exact reconstruction also compared the formulas with separately generated finite universes: 4,096 lists through grade 12, 5,914 permutations through size 7, 626 binary tree shapes through size 7, and 32 objects across finite-product boundary cases. The checks covered counts, dense coordinate coverage and both inverse laws. The list universe was generated by choosing cuts between units, independently of the recursive tail-ranking calculation. The worked coordinates 45 and 13 agreed. Java, Haskell, Mathematica and Rust comparisons were source inspections, not reported executions.

For the musical constructions, exact independent checks compared the rhythm recognition criterion with all positive grid compositions at denominator 8 with five entries and denominator 16 with eight entries: 6,470 candidates. Conditional ranks and motif-occurrence counts were compared with every tree through level 7, including all 4,096 twenty-one-leaf shapes. Both archived Fibonacci orders and a saved notebook example were checked by reconstruction. These checks do not constitute execution of the historical musical software.

The melody formulas were checked against independently sorted signed interval vectors through travel 6 in dimensions 1–4: 1,764 cases. Further checks covered 1,284 pitch/rhythm combinations and all 125 four-note melodies starting at zero on the pitch set $\{-2, -1, 0, 1, 2\}$. Counts, dense coverage, both inverse identities, invalid coordinates and monotonicity were checked. This is mathematical verification of the specified ordering, with no listening results implied.

The added examples were checked independently against 570 expression trees parsed from exhaustively generated postfix strings through size 7, 511 subsets of alphabets through size 8, all 1,100 labeled graphs on at most five vertices, and 979 reduced positive ratios of height at most 40. Seventy-five small topology/alphabet/rule combinations gave 4,243 admitted labeled progression trees. Counts, dense coverage, both inverse maps and boundary rejection were checked against the independently enumerated objects. The recovered pair inverse was checked on 125,751 pairs, large triangular boundaries, both examples in the historical notes and their 42-row table. These are exact mathematical checks; the additional

Rust and historical applications were inspected, not executed.

Testing only $R(U(r)) = r$ on a coordinate prefix cannot reveal an admissible object that U never returns. For a simple example, $U(r) = 2r$ and $R(x) = \lfloor x/2 \rfloor$ pass that identity for every coordinate, yet lose all odd objects if the stated domain is $\mathbb{N}_0$. Object-side tests, explicit boundary cases and a proof of exhaustive decomposition are indispensable. Finite tests support the checked behavior; the general guarantees come from the domains and proofs above.

13 Conclusion

Finite-group decomposition turns counting into reversible coordinates. The same argument constructs mixed-radix products, constrained lists, permutations and recursive shapes: identify a unique group, count its predecessors, and combine coordinates of smaller parts. Finite grading extends the method to countably infinite spaces without placing an infinite block before later objects. The explicit triangular inverse shows how a cumulative count can itself be inverted analytically. Reduced-ratio coordinates show why canonical values must be counted before applying a quotient; labeled progression trees show how local compatibility enters the same construction through completion counts. Subsets, graphs and expression syntax provide compact nonmusical instances.

The musical constructions make that framework concrete. The Fibonacci rhythm criterion determines which anchored duration patterns belong to the family. Motif-conditioned counts then give direct reversible access to extensions of a chosen phrase: four eight-part patterns retain the selected limerick tree, and the occurrence polynomial distinguishes single from repeated appearances at higher levels. For melodies, ordering finite groups by total pitch travel gives a dense coordinate with an exact monotonicity guarantee. Combining it with a finite rhythm family preserves that guarantee, and counting bounded pitch continuations makes an instrument’s range part of the domain.

Its practical strength rests on explicit choices. Equality determines what is being recovered. Counts determine interval boundaries. Compatible component domains and descending recursion make reconstruction valid. Ordering and version make coordinates reproducible. With these conditions in place, direct access, finite uniform sampling and partitioned enumeration follow from one common construction; preference, proximity and efficient constrained counting remain additional choices. A meaningful order starts by specifying what should grow. Here that choice is total pitch movement, with a proved relationship to the coordinate and an explicit question about its perceptual usefulness.

References

1. Wilf, H. S. (1977). A unified setting for sequencing, ranking, and selection algorithms for combinatorial objects. *Advances in Mathematics*, **24**, 281–291. DOI: [10.1016/0001-8708\(77\)90059-7](https://doi.org/10.1016/0001-8708(77)90059-7). [Author’s publication list](#).
2. Duregård, J., Jansson, P., and Wang, M. (2012). Feat: Functional Enumeration of Algebraic Types. *Proceedings of the 2012 Haskell Symposium*, 61–72. DOI: [10.1145/2364506.2364515](https://doi.org/10.1145/2364506.2364515). [Author-hosted paper](#).
3. SIM project (2026a). *sim-lib-rank*, version 0.3.0, in *sim-stream*. [Source at revision d0c1524d2b7c](#).
4. Poetry at Harvard (n.d.). *Key to Poetic Forms*, “Limerick.” [Harvard University](#). Accessed 28 September 2026.
5. Dossou-Olory, A. A. V. (2018). *Leaf-induced subtrees of leaf-Fibonacci trees*. [arXiv:1811.06392](https://arxiv.org/abs/1811.06392).
6. Jacquemard, F., Ycart, A., and Sakai, M. (2017). Generating equivalent rhythmic notations based

- on rhythm tree languages. *Proceedings of the International Conference on Technologies for Music Notation and Representation (TENOR)*, 145–153. DOI: [10.5281/zenodo.924179](https://doi.org/10.5281/zenodo.924179).
7. Temperley, D. (2008). A probabilistic model of melody perception. *Cognitive Science*, **32**, 418–444. DOI: [10.1080/03640210701864089](https://doi.org/10.1080/03640210701864089). [Author-hosted paper](#).
 8. SIM project (2026b). *sim-music*: pitch-ratio coordinates and progression-tree catalogues. [Source at revision eeb0827f2239](#).
 9. SIM project (2026c). *sim-discrete*: finite combinatorial, graph and coefficient-vector spaces. [Source at revision b3cd5eed2a92](#).